

Sample-Efficient Neurosymbolic Deep Reinforcement Learning

Celeste Veronese
University of Verona
Verona, Italy
celeste.veronese@univr.it

Alessandro Farinelli
University of Verona
Verona, Italy
alessandro.farinelli@univr.it

Daniele Meli
University of Verona
Verona, Italy
daniele.meli@univr.it

ABSTRACT

Reinforcement Learning (RL) is a well-established framework for sequential decision-making in complex environments. However, state-of-the-art Deep RL (DRL) algorithms typically require large training datasets and often struggle to generalize beyond small-scale training scenarios, even within standard benchmarks. We propose a neuro-symbolic DRL approach that integrates background symbolic knowledge to improve sample efficiency and generalization to more challenging, unseen tasks. Partial policies defined for simple domain instances, where high performance is easily attained, are transferred as useful priors to accelerate learning in more complex settings and avoid tuning DRL parameters from scratch. To do so, partial policies are represented as logical rules, and online reasoning is performed to guide the training process through two mechanisms: (i) biasing the action distribution during exploration, and (ii) rescaling Q-values during exploitation. This neuro-symbolic integration enhances interpretability and trustworthiness while accelerating convergence, particularly in sparse-reward environments and tasks with long planning horizons. We empirically validate our methodology on challenging variants of gridworld environments, both in the fully observable and partially observable setting. We show improved performance over a state-of-the-art reward machine baseline.

KEYWORDS

Neurosymbolic Reinforcement Learning, Knowledge Transfer, Symbolic Knowledge, Generalization

ACM Reference Format:

Celeste Veronese, Alessandro Farinelli, and Daniele Meli. 2026. Sample-Efficient Neurosymbolic Deep Reinforcement Learning. In *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*, Paphos, Cyprus, May 25 – 29, 2026, IFAAMAS, 9 pages.

1 INTRODUCTION

Deep Reinforcement Learning (DRL) can be successfully applied to solve sequential decision-making problems, offering invaluable benefits for many real-world domains of application, e.g., robotic tasks [13] and sustainability [33], involving complex dynamics, multiple performance objectives, and large observation spaces. However, DRL algorithms still present drawbacks that limit their wide adoption to real systems. Firstly, DRL policies are black-box, which makes them hardly interpretable to humans, affecting trustworthiness and social acceptance [30]. Moreover, one big challenge of DRL

lies in *sample inefficiency* [6], as the agent needs to collect numerous experiences from the environment to build an accurate model and achieve optimality. Sample inefficiency is particularly problematic when scaling and generalizing DRL to environments with longer planning horizons, more sub-goals, and sparse rewards [6] (e.g., larger grids with more objects in gridworlds). Though several approaches have been proposed to mitigate this issue, they either require the availability of a large variety of previous data [1], or rely on specific assumptions on the parametrization of the policy [31]. Heuristic-guided DRL can potentially mitigate the aforementioned issues, but the currently established approaches based on reward shaping or augmentation (reward machines [27]) are practically sample-inefficient and sensitive to the accuracy of heuristics [2].

In this paper, we propose a novel neuro-symbolic approach to improve sampling efficiency of DRL when generalizing to domains with longer planning horizons, more sub-goals, and sparse rewards. Specifically, we adapt interpretable symbolic knowledge from small-scale, easy-to-solve scenarios to guide the training of DRL agents in more challenging settings, in which the neural algorithm obtains poor performance due to sample inefficiency. We include symbolic knowledge as logical specifications (rules) representing an approximation of the policy learned by the agent in simpler domain instances (e.g., small grids with few objects in gridworlds), which require limited exploration. When facing more complex or diverse scenarios, the training of the DRL agent is then enhanced by performing autonomous reasoning on the knowledge transferred from the easier setting, in order to deduce the set of most promising actions given the observation of the agent. We then leverage this knowledge as a planning heuristic to be used directly at the algorithmic level of DRL. In contrast to existing works requiring an exact definition of sub-goal specifications or sub-plans [17], our methodology works even with imperfect symbolic knowledge (e.g., learned from data [8]), and does not require DRL parameter re-tuning when generalizing large domain instances. In more detail, the contributions of this paper are the following:

- we propose a novel neuro-symbolic approach for DRL, which exploits autonomous reasoning on partial logical policy specifications (either handcrafted or acquired in easier settings) to deduce the most promising actions to be taken. We present two different integrations of the symbolic knowledge into the DRL training process, both in exploration and exploitation. Our solution is implemented for the popular class of ϵ -greedy DRL algorithms, which are relevant for the DRL community [20] but are still sample-inefficient [4].
- we exploit an ϵ -decay strategy to balance between the neural and symbolic components without compromising the exploration-exploitation tradeoff of the original DRL algorithm;

- we assess the performance of our methodology in two relevant gridworlds, *OfficeWorld* [27] and *DoorKey* [3], characterized by sparse rewards, multiple sub-goals, and long planning horizons. In particular, we leverage logical heuristics learnt from DRL executions in small grids with few objects, and then assess sampling efficiency and training performance on larger grids with more items (hence, longer planning horizon and more sub-goals) and partially observable scenarios. We show that our methodology is more robust to imperfect partial policies, against an established neuro-symbolic DRL approach based on reward machines [27]. We finally perform an ablation study to evidence the necessary balance between symbolic reasoning and DRL via both the ϵ -decay mechanism and a *confidence* parameter ρ that assesses how much we trust the symbolic knowledge.

2 RELATED WORKS

The problem of sampling efficiency prevents DRL scalability and generalization in the presence of long planning horizons, sparse rewards, and many sub-goals. Initially, ad-hoc algorithms have been developed to face this problem, such as Deep Q-Network (DQN), Rainbow DQN, SAC, etc. [26]. Some approaches then tackled this problem by increasing the variability of the environment at the training stage, possibly with past training information [1] or human intervention [25]. In [14, 18] regularization techniques are employed to prevent overfitting in DRL, which is a major cause for the lack of generalization. Nevertheless, these approaches still require numerous training iterations in large domains; furthermore, the interpretability of the learned policies is still hindered, especially when they are represented by deep neural networks.

The methodology proposed in this paper lies in the neuro-symbolic DRL research area, which combines the abstraction and generalization capabilities of interpretable symbolic (logical) representation and reasoning tools with the inherent advantages of neural approaches when dealing with uncertain data from environmental interaction. The most prominent approach to neuro-symbolic DRL exploits the definition of logical specifications to derive automata, driving the agent towards sub-goals in a hierarchical planning framework [17] or by shaping the reward [5, 8]. However, reward shaping approaches still require many environmental interactions and do not solve the sample-inefficiency, especially with a long planning horizon or with imperfect heuristics [2]. Recent works [22, 24, 29] propose to learn symbolic knowledge from past traces (state-action pairs) of an agent’s executions, instead of handcrafting symbolic knowledge. However, they focused either on the exploration phase only [22, 29], or on a soft initialization of Q-values in tabular RL [24], without comprehensively realizing an efficient and adaptive neuro-symbolic integration for DRL. On the contrary, we propose a novel neuro-symbolic methodology for DRL, which jointly leverages the symbolic knowledge at the algorithmic level, both in exploitation and exploration. Specifically, we consider the popular class of ϵ -greedy DRL algorithms, where exploitation and exploration are neatly separated. We modulate the exploration factor ϵ to probabilistically favour the selection of symbolically entailed actions. This is achieved by biasing the action distribution

in exploration and adaptively re-scaling the Q-values during exploitation, according to ϵ -parameter. Thanks to an ϵ -decay strategy, the early exploration of the agent is more sample-efficient, a fundamental requirement for scalable DRL [15]. At the same time, we progressively modulate the impact of symbolic reasoning over DRL, mimicking the human-like synergy between fast and slow thinking [16].

Finally, our methodology presents some similarities with Statistical Relational Learning (SRL) [21], as proposed in [11]. However, SRL hardly scales to complex domains, requiring the accurate definition of the policy search space [11]. Furthermore, by exploiting the advantages of both symbolic reasoning and DRL, with respect to pure logical learning proposed in [11], our algorithm efficiently generalizes to more challenging domains with longer planning horizons and more sub-goals, where standard DRL algorithms fail.

3 BACKGROUND

We here introduce the relevant background to our methodology, i.e., solving Markov Decision Processes (MDPs) with ϵ -greedy DRL, our testing domains, and the logical framework of Answer Set Programming (ASP) [19], which is the state of the art for logical representation and reasoning on planning problems [23].

3.1 MDPs and Reinforcement Learning

Markov Decision Processes (MDPs) provide a formal framework for planning and decision-making in deterministic and stochastic environments [26]. A MDP is defined by the tuple (S, A, T, R, γ) , where S is the set of states describing the environment; A is the set of possible actions; $T : S \times A \rightarrow \Pi(S)$ is the state transition function, representing the probability of transitioning to state s' from state s when action a is taken; $R : S \times A \rightarrow \mathbb{R}$ is the reward function, providing the immediate reward received after taking action a in state s ; $\gamma \in [0, 1]$ is the discount factor, which determines the present value of future rewards. Planning with an MDP aims at finding an optimal policy $\pi^* : S \rightarrow A$ that maximizes the expected cumulative reward (return) over time. The return, starting from state s and following a policy π , is captured by the value function $V^\pi(s)$:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, \pi \right]$$

In model-free DRL, the agent learns to solve a MDP by interacting with the environment directly, learning an optimal policy and the value function as it performs actions in the environment and collects rewards. Typically, DRL is based on the interleave between exploitation and exploration. During *exploitation*, the agent selects the action according to the currently learned policy and value models; in the *exploration* phase, the agent randomizes its decision in order to explore new regions of the policy space and ultimately improve its learned model. In this paper, we focus on a specific class of DRL algorithms based on ϵ -greedy exploration strategy [20], where typically a random action is chosen with ϵ probability. DQN is the most popular representative of this class of algorithms [26]. It uses a deep neural network (Q-network) to approximate an action-value function $Q(s, a; \theta)$, where s represents the state, a the action, and θ the parameters of the neural network.

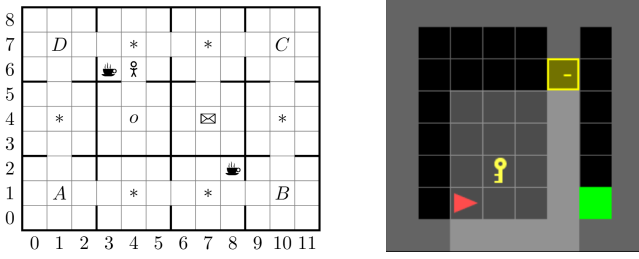


Figure 1: Our testing domains: *OfficeWorld* (left) and *DoorKey* (right).

DQN employs an ϵ -greedy policy with an ϵ -decay strategy to balance exploration and exploitation, by gradually reducing ϵ to favor exploitation as the training progresses. Initially, ϵ is set to a high value, encouraging exploration by selecting random actions. Over time, ϵ is gradually reduced according to a decay schedule, allowing the agent to exploit the learned Q-values more frequently as training progresses. This decay is crucial for the agent to sufficiently explore the environment in the early stages and focus on exploiting its knowledge later.

3.2 Domains

In the **OfficeWorld** domain (Figure 1) [27], the agent is moving on a 9×12 grid, representing various rooms. The state space is given by the set of all locations in the grid (i.e., the agent knows the coordinate of the location it is stepping on), and the observable elements represented by coffee, mail, decorations (marked as * in Figure 1), rooms A, B, C, D and office o. The fully deterministic action space is defined as $A = \{\text{left, right, up, down}\}$. We tested our methodology on the *DeliverCoffeeAndMail* task, which requires bringing both coffee and mail to the office, and on the *PatrolABC* task, in which the agent must visit rooms A, B, and C to complete the task. In both cases, stepping on a decoration ends the episode with a failure.

In the **DoorKey** domain (Figure 1), an agent (red) has to pick up a key to unlock a door (the key and the door must be of the same color) and then get to the green goal square. The environment is discretized in uniform cells, and the state space the agent can observe is a 7×7 portion of the full grid in front of it (unless walls are present), as shown in light gray in Figure 1. Each tile is encoded as a 3-dimensional tuple (object, color, state), indicating, respectively, the object that occupies that cell (e.g. a key, a door or a wall), its color and its state (open or closed, in case the object is a door). The action space for the domain is $A = \{\text{left, right, forward, pickup, open}\}$. Performing actions left and right, the agent turns in the corresponding direction; the forward action makes the agent move one cell ahead in the current direction; pickup is only effective in front of a key and it results in picking the key up; and open can be used by the agent to open a door when in front of it, by using a previously collected key. Both environments are fully deterministic, and a reward of $1 - 0.9 * (\text{step_count}/\text{max_steps})$ is given for completing the task, 0 otherwise.

Both domains are particularly challenging because of the highly sparse reward and the need to coordinate multiple macroactions towards sub-goals (e.g., picking the key before opening the door and reaching the goal) for the winning strategy.

3.3 Answer Set Programming and Reasoning

Answer Set Programming (ASP) [19] represents a planning domain by a sorted signature $\mathcal{D}$, with a hierarchy of symbols defining the alphabet of the domain (variables, constants, and predicates). Logical axioms or rules are then built on $\mathcal{D}$. In this paper, we leverage ASP formalism to represent the background knowledge about the MDP policy, either defined by human experts or learned via DRL in small scenarios. Hence, we consider normal rules, i.e., axioms in the form $h : -b_1, \dots, b_n$, where the body of the rule $\mathcal{B} = b_1 \wedge \dots \wedge b_n$ (i.e. the logical conjunction of the literals) serves as the precondition for the head h . In our setting, body literals represent features generated from the state of the environment (e.g., `samecolor(X, Y)` denoting that two elements are of the same color in *DoorKey*), while the head literal represents an action (e.g., `pickup` in *DoorKey*).

Given an ASP problem formulation P , an ASP solver computes the *answer sets*, i.e., the minimal models satisfying ASP axioms, after all variables have been *grounded* (i.e., assigned with constant values). Answer sets lie in Herbrand base $\mathcal{H}(P)$, defining the set of all possible ground terms that can be formed. In our setting, answer sets contain the feasible actions available to the DRL agent.

4 METHODOLOGY

Our neuro-symbolic DRL strategy (SR-DQN) combines ϵ -greedy DRL exploration and exploitation with Symbolic Reasoning (SR) over logical knowledge, approximating a good partial policy for small and simple domain instances. For simplicity, we frame our methodology in the context of the well-established DQN algorithm; however, it can be easily extended to any other ϵ -greedy DRL approach (e.g. Dueling DQN [32], Rainbow DQN [12]). We now detail the main phases of SR-DQN.

4.1 Logical Representation of the MDP

We begin by representing the MDP domain using the logical formalism of ASP. This involves encoding the state and action spaces as ASP terms. In the ASP signature, we define terms corresponding to environmental features $\mathcal{F}$, which capture essential aspects of the state (e.g., `locked(X)` in the *DoorKey* domain, indicating that door X is locked), as well as terms for actions $\mathcal{A}$ (e.g., `left`, a constant term representing the corresponding action).

To bridge the MDP and the corresponding logical representations, we introduce a *feature map* $F_{\mathcal{F}} : S \rightarrow \mathcal{H}(\mathcal{F})$ and an *action map* $F_{\mathcal{A}} : A \rightarrow \mathcal{H}(\mathcal{A})$, where $\mathcal{H}(\mathcal{F})$ and $\mathcal{H}(\mathcal{A})$ denote the Herbrand bases of $\mathcal{F}$ and $\mathcal{A}$, respectively. More specifically, the feature map $F_{\mathcal{F}}$ translates an MDP state s into a logical description composed of ground terms from $\mathcal{F}$. For example, in the example environment depicted in Figure 1, a specific state s might be represented as $F_{\mathcal{F}}(s) = \text{key}(X), \text{notcarrying}$. Likewise, the action map $F_{\mathcal{A}}$ provides a logical representation of MDP actions using ground terms from $\mathcal{A}$. This part of our methodology relies on the following assumptions:

ASSUMPTION 1. The sets $\mathcal{A}$, $\mathcal{F}$ and the mappings $F_{\mathcal{F}}$ and $F_{\mathcal{A}}$ are known a priori.

This is a standard assumption in the literature [8, 22], and it is considerably weaker than requiring full symbolic task specifications or the existence of a symbolic planner [17]. In particular, $F_{\mathcal{A}}$ can be seen as a straightforward symbolic encoding of the MDP’s action space. Furthermore, we do not assume that $\mathcal{F}$ is *complete*, i.e., it need not contain all task-relevant predicates. This relaxation is justified because our neuro-symbolic integration is robust to imperfect partial policies (see Section 4.3). Thus, we assume that a minimal set of domain predicates, along with a grounding mechanism $F_{\mathcal{F}}$, is either known or can be obtained via automated symbol grounding techniques [28].

ASSUMPTION 2. The action map $F_{\mathcal{A}}$ is surjective.

In other words, to each ground logical predicate corresponds at least one MDP action. This assumption is typically satisfied in practice. For discrete action spaces, distinct predicates can be assigned to each action (i.e., $F_{\mathcal{A}}$ is bijective). In continuous settings, it is possible to define a discretization of the action space, where each grounded action predicate maps to a set of continuous actions. It is also possible to learn this mapping [28].

4.2 Logical Representation of Policy Knowledge

Once the MDP is expressed in ASP formalism, we can represent the background information about the policy in terms of the maps $F_{\mathcal{F}}, F_{\mathcal{A}}$. To this aim, we define a *partial logical policy* $\pi_{ASP} : \mathcal{F} \rightarrow \mathcal{A}$, which maps environmental features to action terms. The logical policy encodes normal rules in the form $a : -f_1, \dots, f_n$, with $f_i \in \mathcal{F}, a \in \mathcal{A}$. For instance, in *DoorKey* domain, π_{ASP} may correspond to the rule:

pickup(X) : $\neg$ key(X), door(Y), samecolor(X, Y).

meaning that the agent should pick up a key if it is of the same colour as the door.

We remark that this knowledge can be inaccurate, e.g., learned from previous example executions [8]. In order to account for the possible inaccuracy of partial policies, we also define a confidence level $\rho \in [0, 1)$ about π_{ASP} .

4.3 Neuro-symbolic training

Algorithm 1 gives a general view of how we integrate symbolic reasoning into DQN. The main components of our methodology are highlighted in red. Our goal is to improve the outcome of ϵ -greedy DRL, by reasoning over the logical policy π_{ASP} in order to efficiently bias the agent towards the most promising actions from background policy knowledge. As in standard DQN, after initializing the Q-network Q and the replay buffer M , we set the exploration rate ϵ to a suitable high initial value ϵ_i (Line 1), in order to favour exploration at the early stages of training, where the agent has not collected enough experience to build an accurate Q-network. During each episode of training, starting at state s_0 (Line 3), at each time step t , the algorithm samples a random number $x \in [0, 1]$ uniformly (Line 5), to decide whether to perform exploitation (Line 6-7) or exploration (Line 8-9), both of which are improved via symbolic reasoning and will be explained in detail in the next subsections. After an action is taken, the agent observes the next state and

Algorithm 1 SR-DQN training loop

Require: Env, maps $F_{\mathcal{F}}, F_{\mathcal{A}}$, partial policy π_{ASP} , max episodes E , max steps T , exploration parameters $\epsilon_i, \epsilon_f, \epsilon_r$, partial policy confidence $\rho \in [0, 1)$

- 1: Initialize replay memory M , $Q(s, a; \theta)$, $\epsilon \leftarrow \epsilon_i$
- 2: **for** episode $e = 1$ to E **do**
- 3: Initialize state s_0
- 4: **for** step $t = 1$ to T **do**
- 5: Uniform sample $x \sim [0, 1]$
- 6: **if** $x \geq \epsilon$ **then**
- 7: $a \leftarrow \text{SR-Exploitation}(s_t, \epsilon, \rho)$
- 8: **else**
- 9: $a \leftarrow \text{SR-Exploration}(s_t, \epsilon, \rho)$
- 10: $s_{t+1}, r \leftarrow \text{Env.step}(a)$
- 11: Store (s_t, a, r, s_{t+1}) in M
- 12: $\theta \leftarrow \text{DQNUdpdate}(\theta)$
- 13: $s_t \leftarrow s_{t+1}$
- 14: **if** s_t is terminal **then**
- 15: **break**
- 16: **if** $\epsilon > \epsilon_f$ **then**
- 17: $\epsilon \leftarrow \text{LinearDecrease}(\epsilon, \epsilon_f, \epsilon_r)$

reward, storing this experience in the replay memory for later training. The Q-network is then updated according to the original DRL algorithm. Finally, once the episode reaches termination (Line 15), the exploration rate ϵ is decreased until the lower bound ϵ_f is reached (Line 17)¹.

The exploration fraction parameter ϵ_r controls how quickly, during the training, ϵ_f must be reached, in terms of the number of episodes. For example, $\epsilon_r = 0.5$ means that $\epsilon = \epsilon_f$ is reached at episode $e = E/2$. In this way, the agent favours exploitation in place of exploration as the training progresses, while also relying more on the knowledge gained by the network and less on the background logical knowledge derived from smaller domains.

We now explain in detail how we employ symbolic reasoning in the different phases of training and then analyze how these changes affect the optimality guarantees of the original DRL algorithm.

Algorithm 2 SR-Exploration

Require: $F_{\mathcal{F}}, F_{\mathcal{A}}, \pi_{ASP}$, current state s_t, ϵ, ρ

- 1: $\mathcal{A}_{\pi_{ASP}} \leftarrow \text{ComputeAnswerSet}(\pi_{ASP}, F_{\mathcal{F}}(s_t))$
- 2: $A_{\pi_{ASP}} \leftarrow F_{\mathcal{A}}^{-1}(\mathcal{A}_{\pi_{ASP}})$
- 3: **if** $A_{\pi_{ASP}} \neq \emptyset$ **then**
- 4: sample $a \sim \text{WeightProb}(A, A_{\pi_{ASP}}, \rho)$
- 5: **else**
- 6: Uniform sample $a \sim A$
- 7: **return** a

4.3.1 Neuro-symbolic exploration. The exploration phase in standard ϵ -greedy approaches consists of picking a uniformly random action from the set A . On the contrary, in SR-Exploration (as shown in Algorithm 2) automated reasoning is performed over π_{ASP} to

¹The linear decrease rule is chosen empirically in our methodology, but more sophisticated strategies can be adopted to reduce ϵ , depending on the specific task [10].

identify the set $\mathcal{A}_{\pi_{ASP}}$ of actions (ground terms in ASP formalism) entailed by the background policy knowledge, given the current set of ground environmental features $F_{\mathcal{F}}(s_t)$ (Line 1). Suggested ASP ground actions are then translated to the MDP action space $A_{\pi_{ASP}} \subseteq A$, by considering the pre-image $F_{\mathcal{A}}^{-1}$ of the action map (Line 2, see Assumption 1). The agent then selects an action from A according to a weighted probability distribution (Line 4), where the weights are defined for each action as follows:

$$w_a = \begin{cases} \rho & \text{if } a \in A_{\pi_{ASP}} \\ 1 - \rho & \text{otherwise} \end{cases} \quad (1)$$

and then normalized, such that $\sum_{a \in A} w_a = 1$. As explained in Section 4.2, ρ represents the level of confidence about the knowledge encoded in π_{ASP} , which may be inaccurate, especially when it is learned [22]. The higher ρ value, the higher the probability that the agent selects an action suggested by background policy knowledge. If $A_{\pi_{ASP}} = \emptyset$ (i.e., π_{ASP} cannot suggest any valuable action at the given state s_t), then a uniformly random action is selected from A as in standard DQN (Line 6).

4.3.2 Neuro-symbolic exploitation. Our approach, shown in Algorithm 3, aims at enhancing the standard DRL exploitation phase by biasing the choice towards the most promising action according to the symbolic knowledge. Given the current state s_t , the agent first queries the Q-network Q to obtain estimated Q-values for all possible actions (Line 1). As in DQN, these Q-values represent the expected return for each action under the current policy. We then employ the ASP policy π_{ASP} in the same way as explained in Section 4.3.1 to compute the set of preferred actions $\mathcal{A}_{\pi_{ASP}}$ (Line 2). Subsequently, the Q-values are rescaled (Line 3) by a factor $k_a = 1 + (\epsilon * w_a)$ for each action, with w_a determined following Equation 1. This is aimed at adjusting the action values within the context of the most promising action set $\mathcal{A}_{\pi_{ASP}}$, according to the confidence parameter ρ . Adding ϵ as an additional rescaling parameter allows the agent to increasingly trust the estimations produced by the neural network as the training proceeds. Finally, the action with the highest rescaled Q-value is selected for execution (Line 4).

Algorithm 3 SR-Exploitation

Require: $F_{\mathcal{F}}, F_{\mathcal{A}}, \pi_{ASP}$, access to current network Q , current state s_t, ϵ, ρ

- 1: $Qvals \leftarrow Q(s_t, a; \theta)$
- 2: $\mathcal{A}_{\pi_{ASP}} \leftarrow \text{ComputeAnswerSet}(\pi_{ASP}, F_{\mathcal{F}}(s_t))$
- 3: $Qvals_{\pi_{ASP}} \leftarrow \text{RescaleQvals}(Qvals, \mathcal{A}_{\pi_{ASP}}, \rho)$
- 4: $a \leftarrow \arg \max_a Qvals_{\pi_{ASP}}$
- 5: **return** a

4.4 Impact of symbolic knowledge on training convergence

The integration of symbolic knowledge we propose is designed to improve the efficiency of DRL exploration while maintaining the stability and empirical convergence behavior of the base algorithms. This balance is achieved by modulating the influence of symbolic guidance through the exploration factor (ϵ_r, ϵ_f) , which gradually decreases as training progresses, and the confidence parameter ρ .

At the beginning of training, when the agent has limited experience and the Q-value estimates are still inaccurate, the symbolic component plays a stronger role in shaping the agent’s behavior. By biasing the action selection toward more promising actions, the agent can more efficiently explore relevant regions of the state space, thereby accelerating early learning. As ϵ_t decays, the contribution of symbolic guidance becomes progressively weaker, allowing the learned value function to increasingly dominate the action selection process. This gradual reduction prevents the logical policy from over-constraining the agent’s behavior or trapping it in suboptimal local minima that may arise from incomplete or imperfect symbolic knowledge. The symbolic component thus acts as a form of guided exploration in the early stages, while the long-term learning dynamics of the underlying DRL algorithm remain unaffected.

4.5 Complexity of symbolic inference in training

At each training step (Lines 7 and 9 of Algorithm 1), the RL agent needs to evaluate whether $F_{\mathcal{F}}(s)$ (i.e. the grounding of the current MDP state) satisfies π_{ASP} , which is a fixed set of non-disjunctive ASP rules. It is known that the grounding part of ASP reasoning is the most computationally intensive [9]. Specifically, let each normal rule $r \in \pi_{ASP}$ include v variables that range over a finite domain of N constants. Denote the finite set of all possible ground instances as $g(r, F_{\mathcal{F}}(s))$. The number of such ground instances is $O(N^v)$. Verifying whether these instances are satisfied by $F_{\mathcal{F}}(s)$ then requires checking, for each ground instance, the truth of every literal in its body. For propositional (ground) non-disjunctive programs, this process is linear in the number of literal occurrences [7]. Therefore, the total computational cost of verifying all grounded instances of rules in π_{ASP} can be expressed as:

$$T_{ASP}(F_{\mathcal{F}}(s), \pi_{ASP}) = O\left(\sum_{r \in \pi_{ASP}} |g(r, F_{\mathcal{F}}(s))| \cdot |\mathcal{B}_r|\right),$$

where $|\mathcal{B}_r|$ denotes the number of literals composing the body of rule r . In our domain representations, each rule references only a small subset of state predicates (e.g., objects within the agent’s view), thus the combinatorial term $|g(r, F_{\mathcal{F}}(s))|$ remains small in practice. In more general and complex settings, we remark that π_{ASP} doesn’t evolve during training and $\mathcal{F}$ is known a priori. Hence, it is possible to compute the full grounding before training starts, thereby eliminating the exponential factor of the complexity. Consequently, the symbolic reasoning over π_{ASP} introduces an overhead which is proportional to the number of variable instantiations actually realized in the current state. For compact rule sets and moderate grounding sizes, this practically results in a negligible increment of the original DQN step time.

5 EMPIRICAL EVALUATION

We evaluate our SR-DQN methodology on the *DoorKey* and *Office-World* domains presented in Section 3.2. These domains are widely used in related literature [8, 11, 27], since they present unique challenges, namely i) sparse reward definition; ii) long planning horizon; iii) the need for optimal strategic coordination towards the achievement of sub-goals. Hence, they represent the ideal benchmark to validate our methodology.

In the following, we primarily evaluate the scalability and sampling efficiency of our method in both domains, increasing the planning horizon and number of sub-goals. To this aim, we compare with a standard DQN baseline and a state-of-the-art reward machine methodology as proposed by [27]. We then perform a thorough ablation study in the *DoorKey* domain (which was the most challenging one in our experiments), to separately investigate the performance of symbolic exploration vs. exploitation, and to assess the impact of relevant parameters of our methodology. Importantly, we tuned the DQN baseline on the easier settings (e.g., maps with one key only in *Doorkey*) and applied it to more challenging scenarios using the same set of hyperparameters. This allowed us to test the capabilities of our approach in achieving generalization without requiring the DRL algorithm to be tuned from scratch.

5.1 Logical Domain Representations and Policies

We now introduce the symbolic knowledge π_{ASP} for our testing domains. Crucially, symbolic knowledge represents partial policies which are learned by the authors of [8, 11] in small-scale domains. Hence, they may fail to generalize to more complex settings, e.g., with more items and sub-goals and a longer planning horizon.

5.1.1 OfficeWorld. We formalize the *OfficeWorld* domain by introducing environmental features $\mathcal{F}$ that represent the known positions of the observables in the map: $\text{coffee}(X)$, $\text{mail}(Y)$, $\text{office}(Z)$. Moreover, we introduce the hasCoffee and hasMail predicates to state that the agent has already picked up that item, and hittingDecoration , which represents the presence of a decoration (that must not be broken) in the agent’s moving direction. Finally, predicate $\text{visited}(X)$ states that the agent has already visited room X . As logical policies, we take the ones learned by [8]. Namely, the policy for the *DeliverCoffee* task:

$$\text{goto}(X) : - \text{coffee}(X), \text{ not hasCoffee}, \quad (2)$$

$$\text{ not hittingDecoration}.$$

$$\text{goto}(X) : - \text{office}(X), \text{ hasCoffee}, \quad (3)$$

$$\text{ not hittingDecoration}.$$

and the one for the *VisitAB* task:

$$\text{goto}(A) : - \text{visited}(\text{NONE}). \quad (4)$$

$$\text{goto}(B) : - \text{visited}(A). \quad (5)$$

where $\mathcal{A} = \{\text{goto}(X)\}$ denotes the action of moving to an item. For Assumption 2, we map this to

$$\text{left} : - \text{goto}(X), \text{ on_left}(X).$$

$$\text{right} : - \text{goto}(X), \text{ on_right}(X).$$

$$\text{forward} : - \text{goto}(X), \text{ straight}(X).$$

adding the necessary body predicates to $\mathcal{F}$. The above specifications suggest that the agent should first pick up the coffee (Rule (2)) and then reach the office (Rule (3)), both without breaking any decoration.

5.1.2 Doorkey. For simplicity, we replicate the ASP formulation of the *Doorkey* domain proposed in [11]. As environmental features $\mathcal{F}$, we define $\text{door}(X)$, $\text{key}(Y)$, $\text{goal}(Z)$ to denote doors, keys and

the goal. We then introduce predicate $\text{locked}(X)$ to state that a door is locked; unlocked to state the intermediate door to the goal is open; $\text{samecolor}(X, Y)$ to denote items (either doors or keys) of the same colour; $\text{carrying}(Y)$ to specify that the agent is carrying an object (key); and notcarrying to specify that the agent is not carrying anything.

We also consider the logical policy learned in [11], in a small 5×5 grid with only one door and key:

$$\text{pickup}(X) : - \text{key}(X), \text{ samecolor}(X, Y), \quad (6)$$

$$\text{door}(Y), \text{ notcarrying}$$

$$\text{open}(X) : - \text{key}(Z), \text{ samecolor}(X, Z), \quad (7)$$

$$\text{door}(X), \text{ locked}(X), \text{ carrying}(Z)$$

$$\text{goto}(X) : - \text{goal}(X), \text{ unlocked} \quad (8)$$

Rule (6) suggests that the agent should pick up a key X if it matches the colour of the door Y and the agent is not currently carrying another key. Then, from Rule (7), the agent can unlock a door X if it is holding a matching-colored key Z , the door X is locked and Z is the correct key for that door. Finally, Rule (8) prescribes that the agent moves towards the goal X if all doors along the path are unlocked.

5.2 Scalability study

We compare SR-DQN against the performance of a standard DQN algorithm (DQN in the figures) and DQN with reward machines (RM-DQN in the figures) as designed in [27]. For reward machines, we test different rewards for state transitions in both *Doorkey* and *OfficeWorld* and keep the best-performing ones in the tuning scenarios. For SR-DQN, we empirically choose $\epsilon_f = \epsilon_r = 0.3$ in *Doorkey* tasks, and $\epsilon_f = 0.05$, $\epsilon_r = 0.1$ in *OfficeWorld* tasks. Since we do not have information about the confidence level of π_{ASP} from [11] and [8], we empirically set $\rho = 0.8$ for both domains. For each method, we evaluate the discounted return² achieved over 5 random seeds.

5.2.1 OfficeWorld. To test the scalability performance of SR-DQN in the *OfficeWorld* domain, we employ the policy learned by [8] in the *DeliverCoffee* and *PatrolAB* tasks as partial policies in the more complex *DeliverCoffeeAndMail* and *PatrolABC* tasks, respectively. In this way, we assess the sampling efficiency of our methodology when generalizing to longer planning horizons and more sub-goals. Figure 2 shows the performance of SR-DQN and the baselines. For both tasks, we tuned the base DQN algorithm to solve the easier setting (i.e. *DeliverCoffee* and *PatrolAB*) and then used the same set of hyperparameters to train all the agents in the more challenging tasks. On average, SR-DQN achieves the highest return by the end of training, also proving to be more stable with a lower standard deviation with respect to DQN in particular. On the other hand, RM-DQN converges more slowly to a lower average return with larger variance, proving the inefficiency of reward augmentation, as theoretically suggested by [2].

5.2.2 DoorKey. For *DoorKey*, we evaluate performance across a range of scenarios with increasing complexity. We begin by scaling

²For RM-DQN, we exclude the additional reward from the plots for a fair comparison.

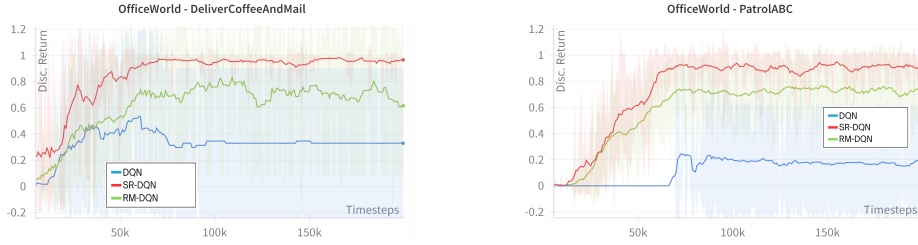


Figure 2: OfficeWorld results on the *DeliverCoffeeAndMail* task (left) and on the *PatrolABC* task (right).

Table 1: Execution times of DQN and SR-DQN algorithms on DoorKey (DK) and OfficeWorld (OW) tasks. Last column shows the time increment introduced by symbolic reasoning.

Domain	Steps	DQN	SR-DQN	Increment
DK 8x8, 1 Key	3M	1h20mins	1h24mins	4 mins (5%)
DK 8x8, 2 Keys	5M	2h10mins	2h15mins	5 mins (3.85%)
DK 8x8, 4 Keys	5M	2h10mins	2h15mins	5 mins (3.85%)
DK 16x16, 1 Key	10M	2h46mins	2h54mins	8 mins (4.82%)
DK 16x16, 2 Keys	10M	2h46mins	2h54mins	8 mins (4.82%)
OW, Deliver	250k	38mins30s	39mins	30s (1.3%)
OW, PatrolABC	250k	40mins	41mins	1min (2.5%)

up the environment from a 5×5 to an 8×8 grid, and simultaneously increase the number of keys (with distinct colors) present in the map, either 2 or 4. We further extend the evaluation to a larger 16×16 grid, considering both one-key and two-key configurations³. In these settings, the agent must also correctly decide which key to pick, depending on the door’s color, in order to finish the task soon and maximize the return. These scenarios provide a compelling demonstration of the benefits of our neuro-symbolic approach, which effectively leverages prior knowledge to generalize to more complex domains. Importantly, none of these larger map configurations were considered in [11], the source of the π_{ASP} policy.

Figure 3 shows the performance of our algorithm and the baselines in a 8×8 map with different amounts of keys in the environment. Even though all algorithms perform similarly in the easiest configuration with just one key (Figure 3, top-left), the SR-DQN algorithm clearly outperforms both the baselines in the more challenging tasks, in which either 2 (Figure 3, top-center) or 4 (Figure 3, top-right) keys are present in the map. Our SR-DQN performs significantly better than both DQN and RM-DQN, showing a higher average return (more than two times the one obtained by DQN). Finally, Figure 3 (bottom line) clearly shows that, even in the bigger 16×16 maps, our algorithm outperforms both the baselines, being the only one able to obtain an acceptable return in both scenarios.

Overall, our neurosymbolic integration demonstrates clear improvements over RM-DQN in environments with longer planning horizons (e.g., multiple keys or larger grids), highlighting the limitations of reward augmentation or shaping in such scenarios.

³We omit the 4-keys configuration in the 16×16 map, as all tested algorithms, including ours, exhibit similarly low performance in this setting.

Finally, in Table 1, we report the execution times of DQN and SR-DQN across all tested tasks. It is evident that the additional overhead introduced by the symbolic inference, indicated in the last row, has a negligible impact on the overall training duration. This demonstrates that integrating symbolic guidance does not compromise the efficiency of the learning process while still providing the benefits evidenced throughout this section.

5.3 Ablation study

Our SR-DQN (Algorithm 1) combines symbolic reasoning both in the exploration (Algorithm 2) and the exploitation (Algorithm 3) phases of DRL, modulated by the decay law of ϵ (Line 20). Together with ρ , the values of ϵ_r and ϵ_f determine the impact of π_{ASP} on the training loop. We now want to investigate in more detail the role of these components independently.

Figure 4 (left) shows the performance of both SR-Exploration (Algorithm 2) and SR-Exploitation (Algorithm 3) when employed as the only symbolic component within the full Algorithm 1. Specifically, as in standard DQN, when disabling SR-Exploration, we sample a uniformly from A during exploration; for SR-Exploitation, we do not rescale Q -values at Line 3 of Algorithm 3. We perform this ablation study on the same 8×8 Doorkey environment with 4 keys, which provides a challenging yet tractable scenario where good performance can still be achieved. Both SR-Exploration and SR-Exploitation alone outperform standard DQN, but SR-Exploitation alone achieves performance very close to that of the full SR-DQN, underscoring the importance of value shaping as a key contributor to overall performance. In contrast, SR-Exploration alone provides a smaller improvement but a faster growth of the return in the very beginning of the training.

In Figure 4 (center), we instead keep the full SR-DQN algorithm, but vary the values of ϵ_f and ϵ_r . Starting from an equal initial value $\epsilon_i = 1$, in this way we modify the decremental behaviour of ϵ , thus varying the impact of the symbolic component of our algorithm. We perform this test in the *DoorKey* environment with 8×8 grid and 4 keys, which represents a great challenge for all the tested baselines but still gives the chance to learn good policies. The optimal curve corresponding to Figure 3 (right) is reported in red ($\epsilon_f = 0.3$, $\epsilon_r = 0.3$). Figure 4 (center) shows that, when decreasing ϵ_f and ϵ_r , thus reducing the impact of the symbolic policy, the training performance decreases significantly. Moreover, a too high ϵ_f value results in unstable policies, thus gaining worse returns.

Finally, Figure 4 (right) shows the performance of SR-DQN under different values of the confidence parameter ρ . Both excessively

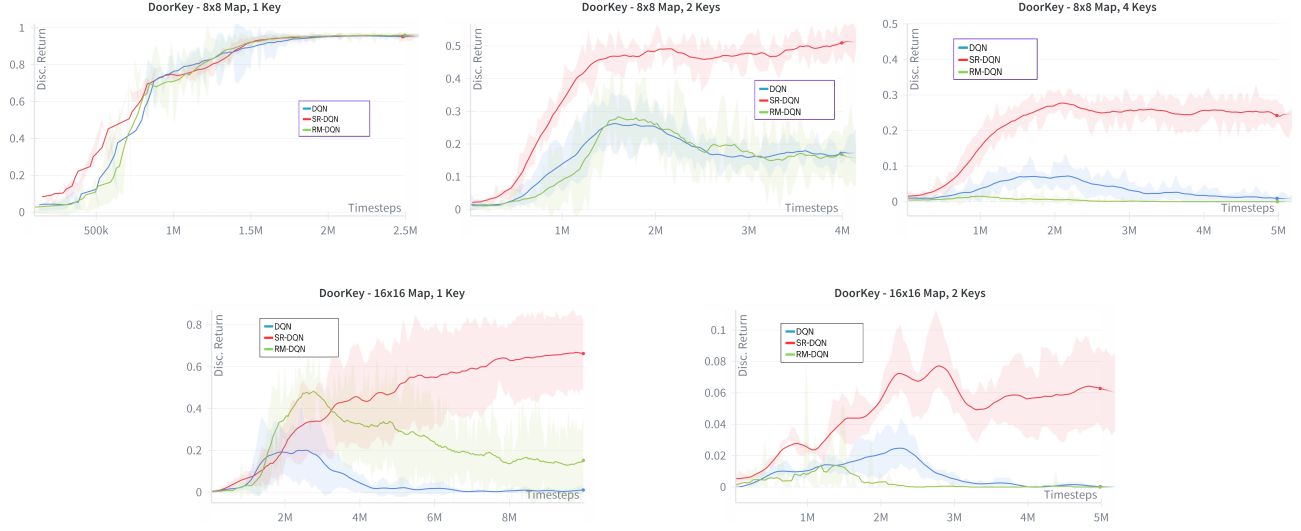


Figure 3: Training results on the DoorKey environment in random maps, varying grid size and number of keys.

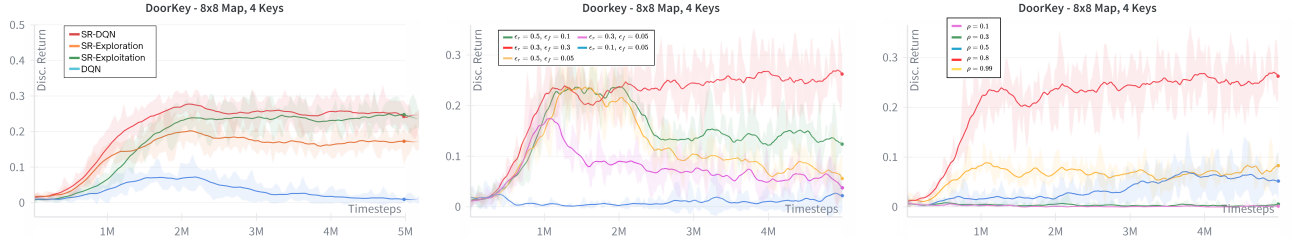


Figure 4: Ablation study over the different components of the SR-DQN algorithm, namely SR-Exploration and SR-Exploitation, compared to the baselines and the full SR-DQN algorithm (left) and training curve of SR-DQN algorithm with either different ϵ_f and ϵ_r (center), or different ρ values (right). All studies are performed on 8×8 DoorKey maps with 4 keys.

low ($\rho \leq 0.5$) and excessively high (ρ close to 1) confidence values lead to suboptimal performance. In the former case, the symbolic component exerts too little influence, preventing the agent from effectively exploiting the structured prior knowledge. In the latter, too much reliance is placed on the symbolic rules, whose accuracy is limited since they are learned from simplified versions of the tasks. This analysis highlights the need for a balanced integration between neural and symbolic components, where ρ , ϵ_r , and ϵ_f regulate the trust in imperfect symbolic knowledge.

6 CONCLUSION AND FUTURE WORK

We presented SR-DQN, a novel neuro-symbolic DRL approach to tackle the problems of scalability and sampling inefficiency in DRL, in environments with long planning horizons, sparse rewards, and multiple sub-goals. Our methodology exploits partial logical policy specifications representing the optimal strategy in easy-to-solve domain instances with limited planning horizon. Then, we perform automated reasoning to entail suggested actions from the logical specifications, biasing both the exploration phase of ϵ -greedy DRL agents and the Q-values produced by the neural component during

training, to encourage the choice of promising symbolic actions. We exploit an ϵ -decay schedule to balance symbolic reasoning and neural learning over time. Importantly, the added symbolic component doesn't represent a significant computational overhead for the original DRL algorithm. We empirically demonstrated the benefits of SR-DQN in two benchmarks, *OfficeWorld* and *DoorKey*, both of which present the challenges mentioned above, as well as partial observability in larger maps. SR-DQN consistently outperformed all the selected baselines (namely, standard DQN, DQN with reward machines, which represent a state-of-the-art technique in neuro-symbolic DRL), also being the only method capable of achieving significant returns in challenging, partially observable *DoorKey* tasks with more items (e.g., multiple keys) and sub-goals, where all tested baselines performed much worse.

In future work, we plan to generalize our methodology to a broader class of DRL algorithms beyond ϵ -greedy strategies. This includes integrating our framework with policy gradient and actor-critic methods. Additionally, we aim to extend our approach to more expressive logical representations, such as temporal or probabilistic logic.

REFERENCES

- [1] Martin Bertran, Natalia Martinez, Mariano Phielipp, and Guillermo Sapiro. 2020. Instance-based generalization in reinforcement learning. *Advances in Neural Information Processing Systems* 33 (2020), 11333–11344.
- [2] Ching-An Cheng, Andrey Kolobov, and Adith Swaminathan. 2021. Heuristic-guided reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 13550–13563.
- [3] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lázcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. 2023. Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks. *CoRR* abs/2306.13831 (2023).
- [4] Chris Dann, Yishay Mansour, Mehryar Mohri, Ayush Sekhari, and Karthik Sridharan. 2022. Guarantees for epsilon-greedy reinforcement learning with function approximation. In *International conference on machine learning*. PMLR, 4666–4689.
- [5] Giuseppe De Giacomo, Luca Iocchi, Marco Favorito, and Fabio Patrizi. 2019. Foundations for restraining bolts: Reinforcement learning with LTLf/LDLf restraining specifications. In *Proceedings of the international conference on automated planning and scheduling*, Vol. 29. 128–136.
- [6] Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468.
- [7] Thomas Eiter, Georg Gottlob, and Nicola Leone. 1997. Abduction from logic programs: Semantics and complexity. *Theoretical Computer Science* 189, 1 (1997), 129–177. [https://doi.org/10.1016/S0304-3975\(96\)00179-X](https://doi.org/10.1016/S0304-3975(96)00179-X)
- [8] Daniel Furelos-Blanco, Mark Law, Anders Jonsson, Krysia Broda, and Alessandra Russo. 2021. Induction and exploitation of subgoal automata for reinforcement learning. *Journal of Artificial Intelligence Research* 70 (2021), 1031–1116.
- [9] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. 2019. Multi-shot ASP Solving with Clingo. *Theory and Practice of Logic Programming (TPLP)* 19, 1 (2019), 27–82. <https://doi.org/10.1017/S1471068418000054>
- [10] Michael Gimelfarb, Scott Sanner, and Chi-Guhn Lee. 2020. Epsilon-BMC: A Bayesian Ensemble Approach to Epsilon-Greedy Exploration in Model-Free Reinforcement Learning. In *Uncertainty in Artificial Intelligence*. PMLR, 476–485.
- [11] Rishi Hazra and Luc De Raedt. 2023. Deep explainable relational reinforcement learning: a neuro-symbolic approach. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 213–229.
- [12] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. 2018. Rainbow: combining improvements in deep reinforcement learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) (AAAI’18/IAAI’18/EAAI’18). AAAI Press, Article 393, 8 pages.
- [13] Julian Ibarz, Jie Tan, Chelsea Finn, Mrinal Kalakrishnan, Peter Pastor, and Sergey Levine. 2021. How to train your robot with deep reinforcement learning: lessons we have learned. *The International Journal of Robotics Research* 40, 4-5 (2021), 698–721.
- [14] Maximilian Igl, Kamil Ciosek, Yingzhen Li, Sebastian Tschitschek, Cheng Zhang, Sam Devlin, and Katja Hofmann. 2019. Generalization in reinforcement learning with selective noise injection and information bottleneck. *Advances in neural information processing systems* 32 (2019).
- [15] Yiding Jiang, J Zico Kolter, and Roberta Raileanu. 2024. On the importance of exploration for generalization in reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [16] Henry Kautz. 2022. The third ai summer: Aai robert s. engelmore memorial lecture. *Ai magazine* 43, 1 (2022), 105–125.
- [17] Harsha Kokel, Sriraam Natarajan, Balaraman Ravindran, and Prasad Tadepalli. 2023. RePreL: a unified framework for integrating relational planning and reinforcement learning for effective abstraction in discrete and continuous domains. *Neural Computing and Applications* 35, 23 (2023), 16877–16892.
- [18] Kimin Lee, Kibok Lee, Jinwoo Shin, and Honglak Lee. 2020. Network Randomization: A Simple Technique for Generalization in Deep Reinforcement Learning. In *Eighth International Conference on Learning Representations, ICLR 2020*. International Conference on Learning Representations.
- [19] Vladimir Lifschitz. 1999. Answer set planning. In *Logic Programming and Non-monotonic Reasoning: 5th International Conference, LPNMR’99 El Paso, Texas, USA, December 2–4, 1999 Proceedings* 5. Springer, 373–374.
- [20] Fanghui Liu, Luca Viano, and Volkan Cevher. 2022. Understanding deep neural function approximation in reinforcement learning via ϵ -greedy exploration. *Advances in Neural Information Processing Systems* 35 (2022), 5093–5108.
- [21] Giuseppe Marra, Sebastijan Dumančić, Robin Manhaeve, and Luc De Raedt. 2024. From statistical relational to neurosymbolic artificial intelligence: A survey. *Artificial Intelligence* (2024), 104062.
- [22] Daniele Meli, Alberto Castellini, and Alessandro Farinelli. 2024. Learning logic specifications for policy guidance in pomdps: an inductive logic programming approach. *Journal of Artificial Intelligence Research* 79 (2024), 725–776.
- [23] Daniele Meli, Hirenkumar Nakawala, and Paolo Fiorini. 2023. Logic programming for deliberative robotic task planning. *Artificial Intelligence Review* 56, 9 (2023), 9011–9049.
- [24] Sarath Sreedharan and Michael Katz. 2023. Optimistic exploration in reinforcement learning using symbolic model estimates. *Advances in Neural Information Processing Systems* 36 (2023), 34519–34535.
- [25] DJ Strouse, Kevin McKee, Matt Botvinick, Edward Hughes, and Richard Everett. 2021. Collaborating with humans without human data. *Advances in Neural Information Processing Systems* 34 (2021), 14502–14515.
- [26] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [27] R. Toro Icarte, T. Q. Klassen, R. A. Valenzano, and S. A. McIlraith. 2018. Using Reward Machines for High-Level Task Specification and Decomposition in Reinforcement Learning. In *Proceedings of the International Conference on Machine Learning (ICML)*. 2112–2121.
- [28] Elena Umili, Roberto Capobianco, and Giuseppe De Giacomo. 2023. Grounding LTLf specifications in image sequences. In *Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning*, Vol. 19. 668–678.
- [29] Celeste Veronese, Daniele Meli, and Alessandro Farinelli. 2024. Online Inductive Learning from Answer Sets for Efficient Reinforcement Learning Exploration. In *Proceedings of the 3rd International Workshop on Hybrid Models for Coupling Deductive and Inductive Reasoning (HYDRA)*. Currently under publication.
- [30] George A Vouros. 2022. Explainable deep reinforcement learning: state of the art and challenges. *Comput. Surveys* 55, 5 (2022), 1–39.
- [31] Huan Wang, Stephan Zheng, Caiming Xiong, and Richard Socher. 2019. On the generalization gap in reparameterizable reinforcement learning. In *International Conference on Machine Learning*. PMLR, 6648–6658.
- [32] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Van Hasselt, Marc Lanctot, and Nando De Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48* (New York, NY, USA) (ICML’16). JMLR.org, 1995–2003.
- [33] Maddalena Zuccotto, Alberto Castellini, Davide La Torre, Lapo Mola, and Alessandro Farinelli. 2024. Reinforcement learning applications in environmental sustainability: a review. *Artificial Intelligence Review* 57, 4 (2024), 88.